The Name of the Title Is Hope

Ben Trovato
VUnet ID
larst@affiliation.org

Lars Thørväld
VUnet ID
larst@affiliation.org

Valerie Béranger
VUnet ID
larst@affiliation.org

Charles Palmer
VUnet ID
cpalmer@prl.com

Figure 1. Seattle Mariners at Spring Training, 2010.

Abstract

Concise summary (typically 150–300 words). Start by identifying a specific challenge in a problem domain and explain how your proposed Domain-Specific Language (DSL) addresses it more effectively than general-purpose languages (GPLs). Briefly mention the core abstractions, the implementation strategy (e.g., code generation or interpretation), and your most significant evaluation finding—such as a specific performance gain or a reduction in lines of code.

1 Introduction

Context: Delineate the "semantic gap" in current engineering practices for your domain.

Problem Statement: Explicitly state why existing tools or GPLs are insufficient (e.g., excessive boilerplate, lack of domain-aware verification).

Contributions: Provide a clear list of what this paper adds to the field of software language engineering. Typical contributions include:

- The design and design principles of the language L.
- A formal specification of its syntax and semantics.
- The implementation of the compiler/interpreter and supporting IDE tooling.
- An empirical evaluation proving its efficacy (e.g., case studies or benchmarks).

Vrije Universiteit Amsterdam, SLE Seminar, March 16, 2026
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Domain Analysis

This section justifies your language design through a systematic study of the problem area.

Stakeholders and Use Cases: Identify who will use the language and for what tasks (e.g., documentation vs. code generation).

Requirements: List the technical requirements for the language, such as incrementality, persistence, or correctness.

Concepts and Operations: Use established methodologies like Feature-Oriented Domain Analysis (FODA) to cluster domain knowledge into first-class semantic notions that the language will support.

3 Language Design

3.1 Abstract Syntax

Describe the logical structure of the language, typically using an Algebraic Data Type (ADT) or metamodel (e.g., Ecore or MOF). Explain how this metamodel captures the domain concepts identified during analysis.

3.2 Concrete Syntax

Detail the notation through which users interact with the language (textual, graphical, or projectional). For textual syntaxes, use formal grammars like BNF or EBNF. Explain how your notation minimizes cognitive load by using domain-specific vocabulary.

4 Formal Semantics

4.1 Static Semantics

Define the rules checked during the program analysis, such as type systems, scoping, and well-formedness constraints. Precise formal notation, such as inference rules for type judgments, is highly recommended.

4.2 Dynamic Semantics

Describe the execution behavior of your language constructs. Choose the approach that matches your implementation:

Translational Semantics: If your DSL uses code generation, describe the mapping from your concrete syntax to the target language (e.g., Java, C++, SQL) and any domain-specific optimizations applied during translation.

Operational/Denotational Semantics: If your DSL uses direct interpretation, describe the operational rules or mathematical foundation defining the language's meaning.

5 Implementation

Discuss the technical development of your DSL, often using a language workbench. If not, explain why and your motivation behind this design decision.

Workbench Infrastructure: Identify the tools used (e.g., Spoofax, Xtext, MontiCore, or MPS).

System Architecture: Describe the split of the front-end (parsing and semantic analysis) and back-end (code generation or interpretation).

Tooling Support. : Mention generated IDE features like syntax highlighting, auto-completion, and real-time error reporting.

6 Evaluation

The evaluation must demonstrate that your DSL delivers on its core promises.

Case Studies: Re-implement a known complex system to demonstrate how domain abstractions simplify the solution. Report quantitative improvements, such as a reduction factor in lines of code.

Performance Benchmarks: For performance-critical DSLs, provide execution time or memory usage data compared with hand-coded GPL baselines. *User Studies:* If targeting "lay programmers" or domain experts, report on usability metrics such as task completion time and error rates.

7 Related Work

Classify and critically analyze related work based on implementation strategies (internal vs. external DSLs) or notation types. Explicitly compare your language's abstractions against existing libraries or specialized languages to highlight your unique contribution.

8 Conclusion and Future Work

Summarize the findings and discuss the implications of your work. Identify future research directions, such as scaling the DSL, language evolution, or integrating AI-assisted design.

9 Individual Contributions

Describe in detail what was the contribution of each of the team members to the development of the project including: design, documentation, implementation, and presentation.

References